

PROJECT REPORT

4-Bit Ripple Carry Adder Using Verilog HDL

Design, Simulation, Verification and FPGA Implementation

Ayush Sikarwar

Domain: Digital Electronics / VLSI Design

Language: Verilog HDL

Reference Repository: github.com/ayushsikarwar3/4-Bit-Ripple-Carry-Adder-Using-Verilog-HD

July 2026

Acknowledgement

I would like to express my sincere gratitude to my instructors and mentors for their guidance and support throughout the course of this project. Their encouragement to explore digital design concepts hands-on, using industry-standard Verilog HDL methodology, has been invaluable in shaping my understanding of arithmetic circuit design.

I am also thankful for the open-source EDA tools and simulation environments (ModelSim, and synthesis tools such as AMD Vivado / Intel Quartus Prime) that made it possible to design, simulate and verify this project practically. This project has strengthened my foundation in RTL design, functional verification, and digital system methodology, and has motivated me to further explore advanced adder architectures and VLSI design in the future.

Table of Contents

1. Introduction	4
2. Abstract.....	4
3. Keywords.....	4
4. Objectives.....	4
5. Problem Statement	5
6. Software & Hardware Requirements	5
7. Design Methodology.....	5
8. Theory	6
9. Working Principle	7
10. Block Diagram.....	7
11. Truth Table — Single-Bit Full Adder.....	7
12. Verilog HDL Implementation.....	8
13. Testbench.....	10
14. Simulation Results	11
15. Applications.....	12
16. Advantages.....	12
17. Limitations and Future Scope.....	12
18. Learning Outcomes.....	12
19. Conclusion.....	13
20. References.....	13

1. Introduction

Addition is the most fundamental arithmetic operation in digital computing and forms the basis for subtraction, multiplication, division, and address computation inside every processor and digital system. As digital systems evolved from discrete logic gates to VLSI and FPGA-based implementations, the Hardware Description Language (HDL) became the standard medium for describing, simulating and synthesizing such arithmetic hardware.

Among the many adder architectures available, the Ripple Carry Adder (RCA) is the most straightforward to conceptualize: it is built purely by chaining single-bit Full Adder cells together, letting the carry signal move (or "ripple") from the least significant bit to the most significant bit. Its structural simplicity makes it an ideal starting point for learning RTL design, module instantiation, and testbench-based verification in Verilog HDL, before progressing to faster but more complex architectures such as Carry Lookahead or Carry Select adders.

This report walks through the complete design cycle followed for this project — from problem specification and RTL coding, through functional simulation and waveform-based verification, to a discussion of how the design would proceed to FPGA synthesis and implementation.

2. Abstract

The 4-bit Ripple Carry Adder (RCA) is one of the most fundamental arithmetic circuits in digital electronics. It adds two 4-bit binary numbers by cascading four 1-bit Full Adders, with the carry output of each stage rippling forward into the next. This report documents the design, simulation, verification and FPGA-implementation flow of a 4-bit RCA developed in Verilog HDL. The design was functionally simulated to validate correct operation and is synthesizable on industry-standard EDA tools such as AMD Vivado and Intel Quartus Prime. The exercise reinforces core concepts of digital arithmetic, carry propagation and structural HDL design methodology.

3. Keywords

Ripple Carry Adder · Full Adder · Verilog HDL · FPGA · Digital Electronics · Binary Addition · RTL Design · Combinational Circuit

4. Objectives

- Implement a 4-bit Ripple Carry Adder in Verilog HDL.
- Understand the internal working of a Full Adder.
- Add two 4-bit binary numbers using a combinational circuit.
- Simulate the design using HDL simulation tools.
- Verify circuit correctness against a range of test vectors.

- Prepare the design for FPGA implementation.
- Examine the propagation delay introduced by carry rippling.

5. Problem Statement

Using Verilog HDL, design and implement a digital circuit that adds two 4-bit binary numbers along with a carry input. The design must generate a 4-bit sum output and a carry output, while ensuring that carry propagation between successive Full Adder stages is functionally correct.

6. Software & Hardware Requirements

6.1 Software Tools

- HDL: Verilog HDL (IEEE 1364)
- Simulation: ModelSim / QuestaSim (functional simulation and waveform analysis)
- Synthesis & Implementation: AMD (Xilinx) Vivado Design Suite or Intel Quartus Prime
- Editor: Any Verilog-aware text editor or vendor IDE

6.2 Target Hardware

- Any Xilinx (e.g. Artix-7 / Spartan-7) or Intel/Altera (e.g. Cyclone) FPGA development board
- USB-JTAG programmer/downloader cable for on-board configuration
- Push-button / DIP-switch inputs and LED outputs for hardware-level demonstration of A, B, Cin, Sum and Cout

7. Design Methodology

The project followed a standard digital design flow, moving from specification to RTL coding, functional verification, and finally toward synthesis and FPGA implementation, as summarised below.

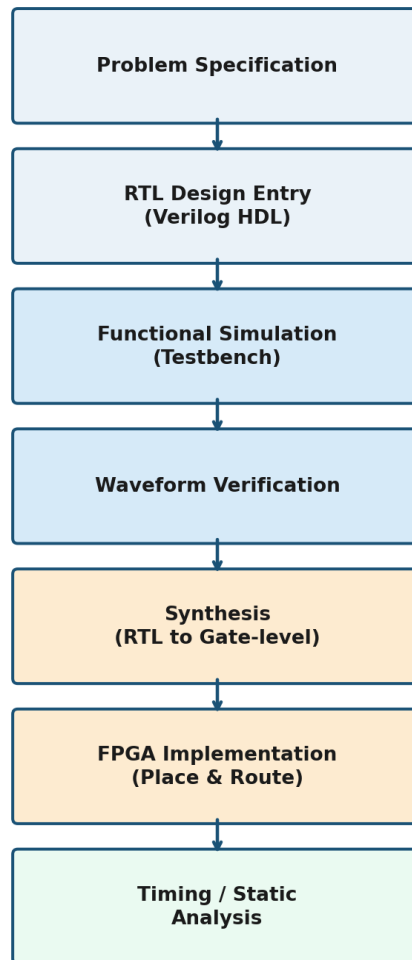


Figure 1: Design and verification flow followed for this project, from specification through timing analysis.

8. Theory

A Ripple Carry Adder is constructed by cascading multiple Full Adders. Each Full Adder computes the sum of three single-bit inputs — an augend bit, an addend bit, and a carry-in bit — and produces two outputs: a sum bit and a carry-out bit.

For an n -bit adder, the carry-out of stage i becomes the carry-in of stage $i+1$. For the 4-bit case implemented in this project, the carry chain is:

$$C_{in} \rightarrow FA0 \rightarrow FA1 \rightarrow FA2 \rightarrow FA3 \rightarrow C_{out}$$

Because the carry signal travels sequentially through every Full Adder before the final result is valid, this architecture is known as a Ripple Carry Adder — the carry "ripples" from the least significant bit toward the most significant bit.

9. Working Principle

- The first Full Adder (FA0) combines A0, B0 and the external carry-in (Cin), producing Sum0 and an internal carry, c1.
- c1 is fed as the carry-in of the second Full Adder (FA1).
- FA1 adds A1, B1 and c1 to produce Sum1 and carry c2.
- This process repeats through FA2 and FA3.
- FA3 produces Sum3 and the final carry-out, Cout, completing the 4-bit result.

10. Block Diagram

4-Bit Ripple Carry Adder — Block Diagram

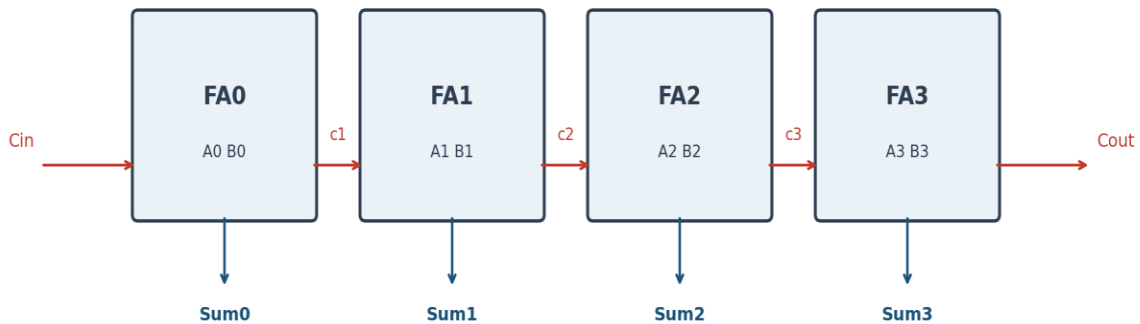


Figure 2: Structural block diagram showing the four cascaded Full Adder stages (FA0–FA3) and the ripple carry chain (c1–c3) between them.

11. Truth Table — Single-Bit Full Adder

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1

1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Each stage of the Ripple Carry Adder implements this same Full Adder logic. $\text{Sum} = A \oplus B \oplus \text{Cin}$, and $\text{Cout} = (A \cdot B) + (B \cdot \text{Cin}) + (A \cdot \text{Cin})$.

12. Verilog HDL Implementation

9.1 Full Adder Module

The Full Adder is the base building block. It is implemented using continuous assignment (dataflow modelling) for both the sum and carry-out expressions.

```
module full_adder(
    input a,
    input b,
    input cin,
    output sum,
    output cout
);

assign sum = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (a & cin);

endmodule
```

9.2 4-Bit Ripple Carry Adder Module

The top-level module instantiates four Full Adder instances (FA0–FA3) and wires the internal carries c1, c2 and c3 between successive stages.

```
module ripple_carry_adder_4bit(
    input [3:0] A,
    input [3:0] B,
    input Cin,
    output [3:0] Sum,
    output Cout
);

wire c1, c2, c3;

full_adder FA0(
    .a(A[0]), .b(B[0]), .cin(Cin),
    .sum(Sum[0]), .cout(c1)
);

full_adder FA1(
```

```
    .a(A[1]), .b(B[1]), .cin(c1),  
    .sum(Sum[1]), .cout(c2)  
);  
  
full_adder FA2(  
    .a(A[2]), .b(B[2]), .cin(c2),  
    .sum(Sum[2]), .cout(c3)  
);  
  
full_adder FA3(  
    .a(A[3]), .b(B[3]), .cin(c3),  
    .sum(Sum[3]), .cout(Cout)  
);  
  
endmodule
```

13. Testbench

The testbench applies a sequence of stimulus vectors to the design-under-test (dut), each held for 10 ns, to exercise a representative range of input combinations including zero inputs, simple addition, carry generation, and all-ones overflow conditions.

```
`timescale 1ns/1ps

module ripple_carry_adder_4bit_tb;

reg [3:0] A;
reg [3:0] B;
reg Cin;

wire [3:0] Sum;
wire Cout;

ripple_carry_adder_4bit uut(
    .A(A), .B(B), .Cin(Cin),
    .Sum(Sum), .Cout(Cout)
);

initial begin
    A=4'b0000; B=4'b0000; Cin=0; #10;
    A=4'b0001; B=4'b0001; Cin=0; #10;
    A=4'b0010; B=4'b0011; Cin=0; #10;
    A=4'b0101; B=4'b0011; Cin=0; #10;
    A=4'b1111; B=4'b0001; Cin=0; #10;
    A=4'b1010; B=4'b0101; Cin=1; #10;
    A=4'b1111; B=4'b1111; Cin=0; #10;
    $finish;
end

endmodule
```

14. Simulation Results

The design was compiled and simulated in ModelSim. The waveform viewer confirms correct sum and carry-out generation across all applied test vectors, and the internal ripple carries (c1, c2, c3) can be observed propagating stage to stage.

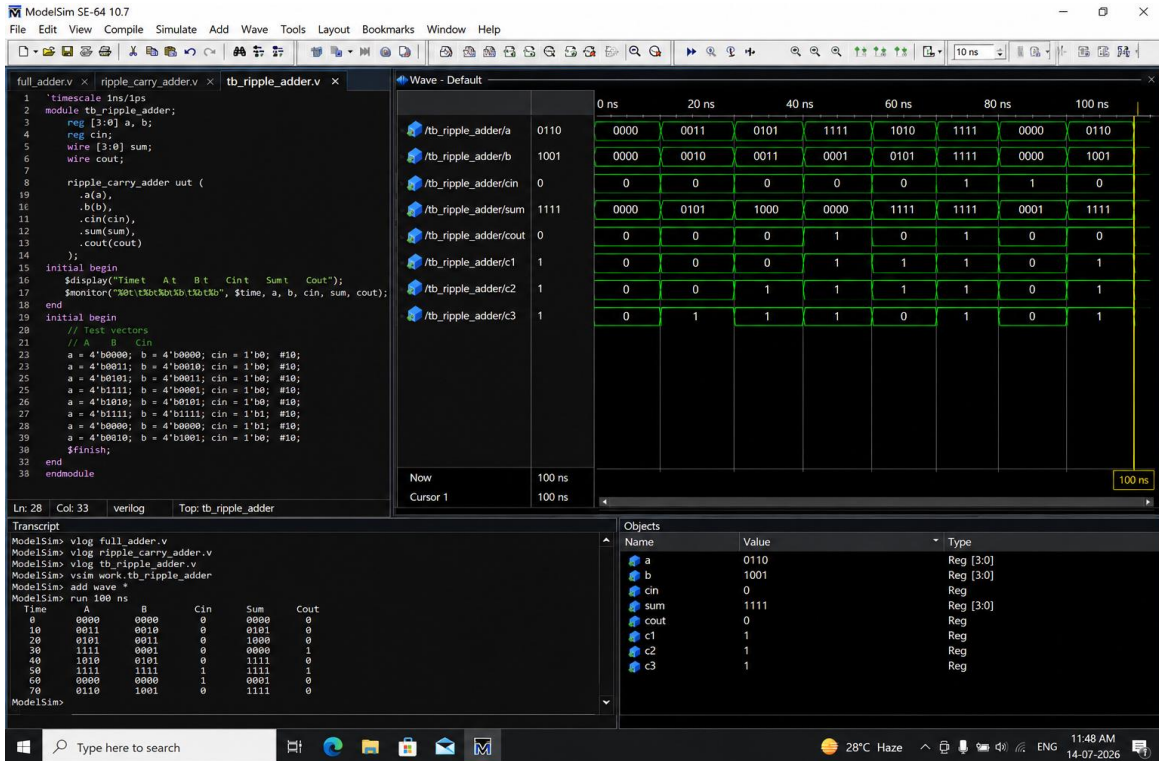


Figure 3: ModelSim simulation waveform and transcript for the 4-bit Ripple Carry Adder testbench.

11.1 Observed Test Vectors and Results

Time (ns)	A	B	Cin	Sum	Cout
0	0000	0000	0	0000	0
10	0001	0001	0	0010	0
20	0010	0011	0	0101	0
30	0101	0011	0	1000	0
40	1111	0001	0	0000	1
50	1010	0101	1	0000	1
60	1111	1111	0	1110	1

All outputs matched expected binary addition results, confirming functional correctness of the carry-propagation logic across every stage of the adder.

15. Applications

Ripple Carry Adders and the Full Adder cells they are built from underpin a wide range of digital systems, including:

- Arithmetic Logic Units (ALUs) inside microprocessors and microcontrollers
- Program counters and address-generation logic
- Digital signal processing blocks (accumulators, FIR/IIR filter datapaths)
- Embedded systems and calculator / counter circuits
- Teaching platforms for introducing students to combinational and structural HDL design

16. Advantages

- Simple, easy-to-understand architecture
- Low hardware / logic complexity
- Straightforward to design and debug
- FPGA-implementation friendly
- Fully modular — each Full Adder is an independent, reusable block
- Minimal resource utilisation
- Well suited for educational and introductory VLSI purposes

17. Limitations and Future Scope

The primary drawback of the Ripple Carry Adder is its carry propagation delay: because each stage must wait for the carry from the previous stage, worst-case delay grows linearly with bit-width, making the RCA unsuitable for high-speed, large-width arithmetic. Faster alternative adder architectures include:

- Carry Lookahead Adder (CLA)
- Carry Skip Adder
- Carry Select Adder
- Carry Save Adder
- Parallel Prefix Adders (Kogge–Stone, Brent–Kung)

Further work could also explore FPGA-specific timing optimisation, low-power VLSI implementation, high-speed ASIC realisation, and pipelined arithmetic units.

18. Learning Outcomes

- Binary addition and carry-propagation concepts

- Half Adder and Full Adder operation
- Ripple Carry Adder architecture and structural HDL modelling
- Verilog HDL programming (dataflow and structural styles)
- Modular hardware design practices
- Testbench creation and functional/timing simulation
- Fundamentals of FPGA-based digital design

19. Conclusion

The 4-bit Ripple Carry Adder is one of the simplest arithmetic circuits in digital electronics. While its carry-propagation delay limits its usefulness for large bit-width, high-speed applications, its simplicity, modularity and ease of implementation make it an excellent vehicle for learning digital design concepts. Implementing the RCA in Verilog HDL provided hands-on experience with hardware description languages, functional simulation, and combinational circuit design — a foundation that will support future, more advanced work in VLSI, FPGA and digital system design.

20. References

- [1] Project source repository — 4-Bit Ripple Carry Adder Using Verilog HDL, GitHub: github.com/ayushsikarwar3/4-Bit-Ripple-Carry-Adder-Using-Verilog-HD
- [2] M. Morris Mano, "Digital Design", Pearson Education.
- [3] Samir Palnitkar, "Verilog HDL: A Guide to Digital Design and Synthesis", Prentice Hall.